

Universidad Mayor Real y Pontificia de San Francisco

Xavier de Chuquisaca

Facultad de Ciencias y Tecnología

Ingeniería en Ciencias de la Computación

SIS306 - Examen Final

SIS306 - Bases de Datos III

Docente: Ing. Edgar T. Espinoza Rodríguez

Estudiantes:

Ávila Serrano Christian Ángel

Peñaranda Villarroel Hernán Isaac

Limachi Villarroel Alan Nicolás

Fecha de entrega: Martes, 23 de junio de 2026

Diseño, transformación e implementación de una base de datos en

Oracle 19c para la gestión odontológica

Introducción

El presente informe desarrolla la documentación técnica de una base de datos para la gestión de procesos odontológicos. El dominio considerado incluye registro de pacientes, asignación de carpetas clínicas, programación de citas, atención mediante odontogramas, tratamientos, planes de pago, registro de pagos, compras de suministros e inventario.

El trabajo toma como referencia documental una tesis previa sobre el desarrollo de un sistema de información web para el consultorio odontológico LALYSDENT. En dicha tesis se documenta un sistema web implementado con Laravel, MySQL y arquitectura MVC. A diferencia de esa investigación, el presente informe no se enfoca en el frontend ni en la aplicación web, sino en el diseño, transformación, implementación y validación de la base de datos.

La propuesta fue desarrollada en etapas. Primero, se definieron los requisitos y supuestos del dominio. Luego, se elaboró un modelo conceptual en notación Chen usando diagrams.net. Posteriormente, se realizó la transformación al modelo relacional y se construyó un modelo lógico Crow's Foot en ER/Studio 2003.

Finalmente, se implementó el diseño físico en Oracle Database 19c, ejecutado en Docker y administrado mediante DBeaver.

El documento incluye también un diccionario de datos completo, validaciones ejecutadas sobre Oracle 19c y una comparación técnica entre el enfoque MySQL/Laravel documentado en la tesis base y la transformación realizada hacia Oracle 19c.

Análisis de requisitos

Alcance del proyecto

El proyecto se concentra en la base de datos como producto principal del examen final. Aunque el sistema puede contar con un frontend, la documentación se limita al diseño, implementación y validación de la base de datos.

El alcance incluye análisis de requisitos, modelo conceptual, transformación relacional, modelo lógico, diseño físico, diccionario de datos, validaciones funcionales y comparación con el enfoque MySQL/Laravel documentado en la tesis base.

Entidades principales

A partir del dominio se identificaron las siguientes entidades: asistente, folder, doctor, tratamiento, proveedor, suministro, paciente, cita, odontograma, detalle de odontograma, plan de pago, pago, compra, detalle de compra e inventario.

Procesos cubiertos

La base de datos cubre cuatro procesos principales: registro y citas, atención odontológica, finanzas y pagos, y compras e inventario.

Modelo conceptual en notación Chen

El modelo conceptual fue elaborado en Draw.io usando notación Chen estricta. En los diagramas se usaron rectángulos para entidades, rombos para relaciones, elipses para atributos, atributos subrayados para identificadores, líneas dobles para participación total y cardinalidad mínima–máxima junto a los conectores. No se modelaron entidades débiles, porque las entidades principales poseen identificador propio. Para mejorar la legibilidad, el modelo se dividió en una vista general y cuatro vistas por módulo.

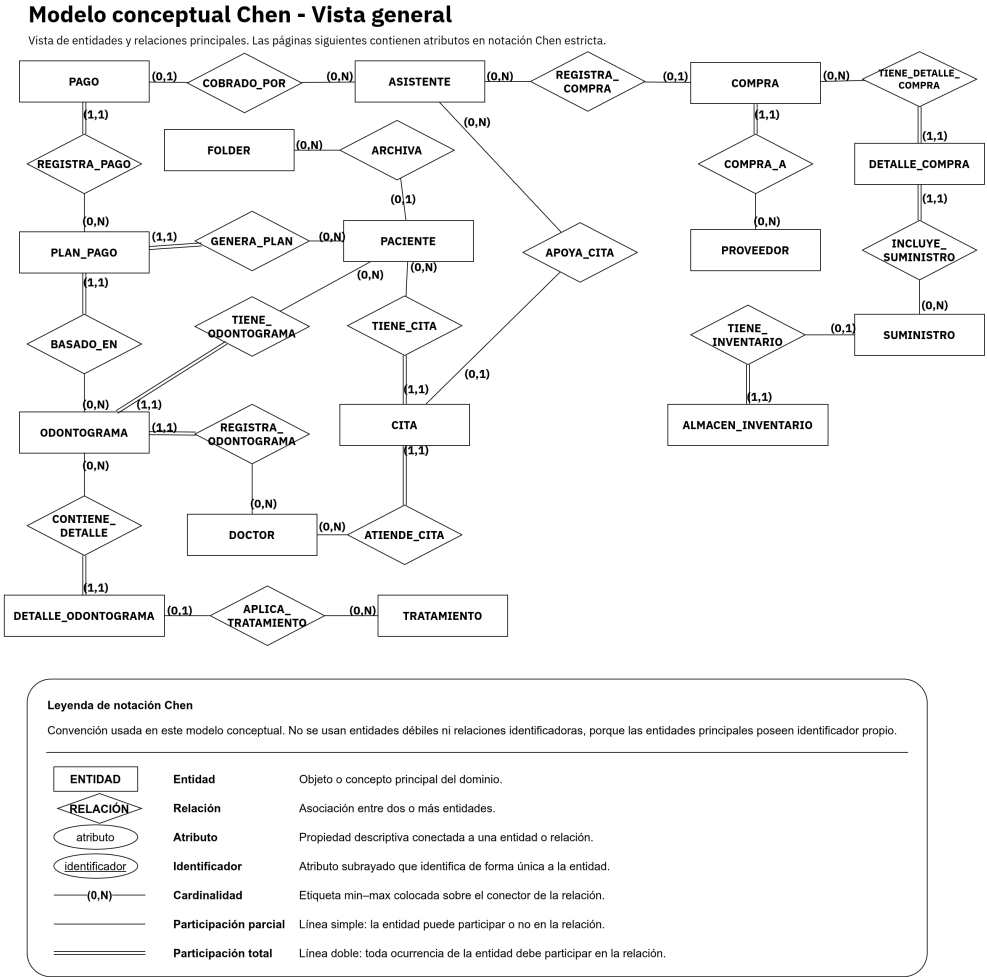


Figura 1

Modelo conceptual Chen: vista general.

Modelo conceptual Chen - Registro y citas

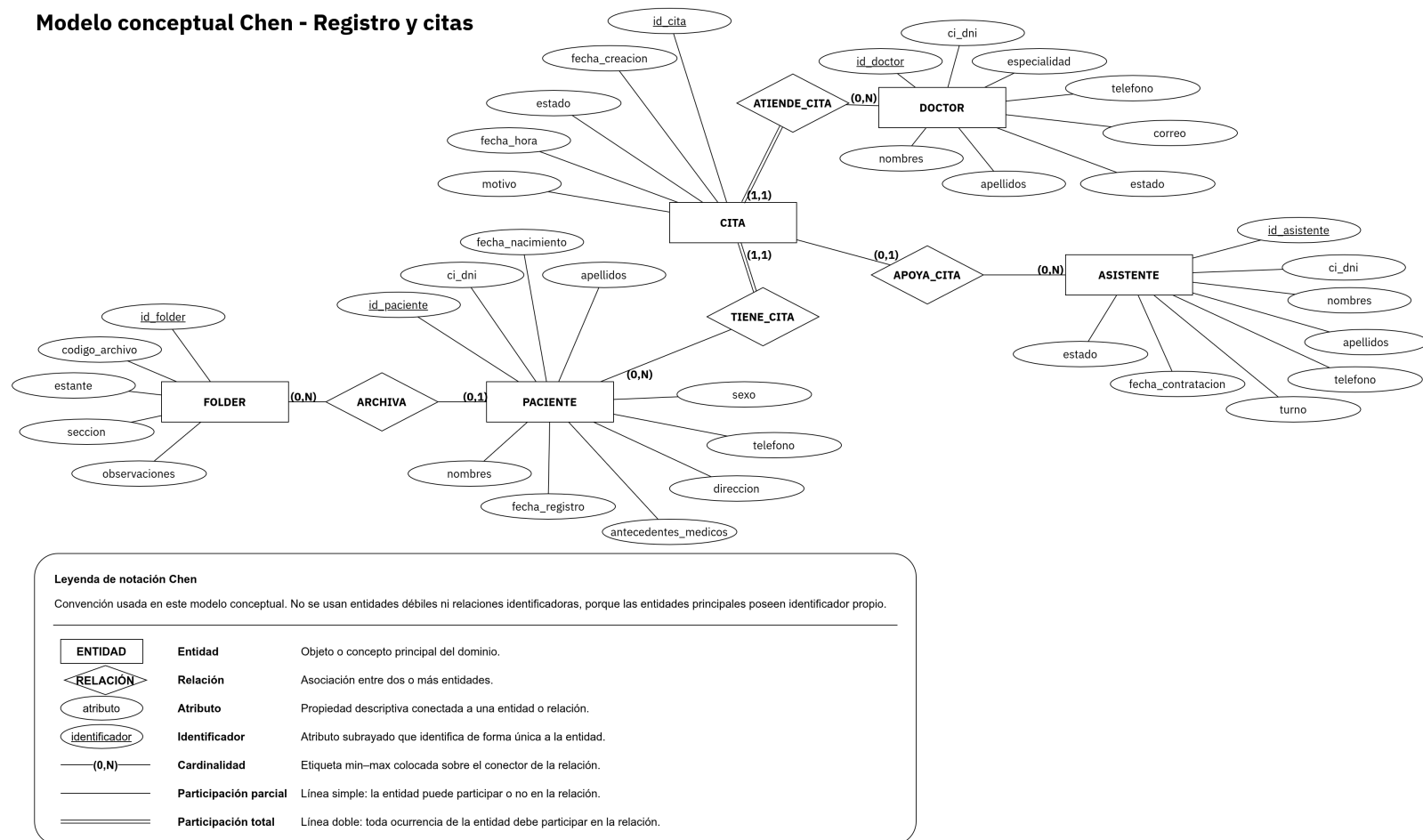


Figura 2

Modelo conceptual Chen: registro y citas.

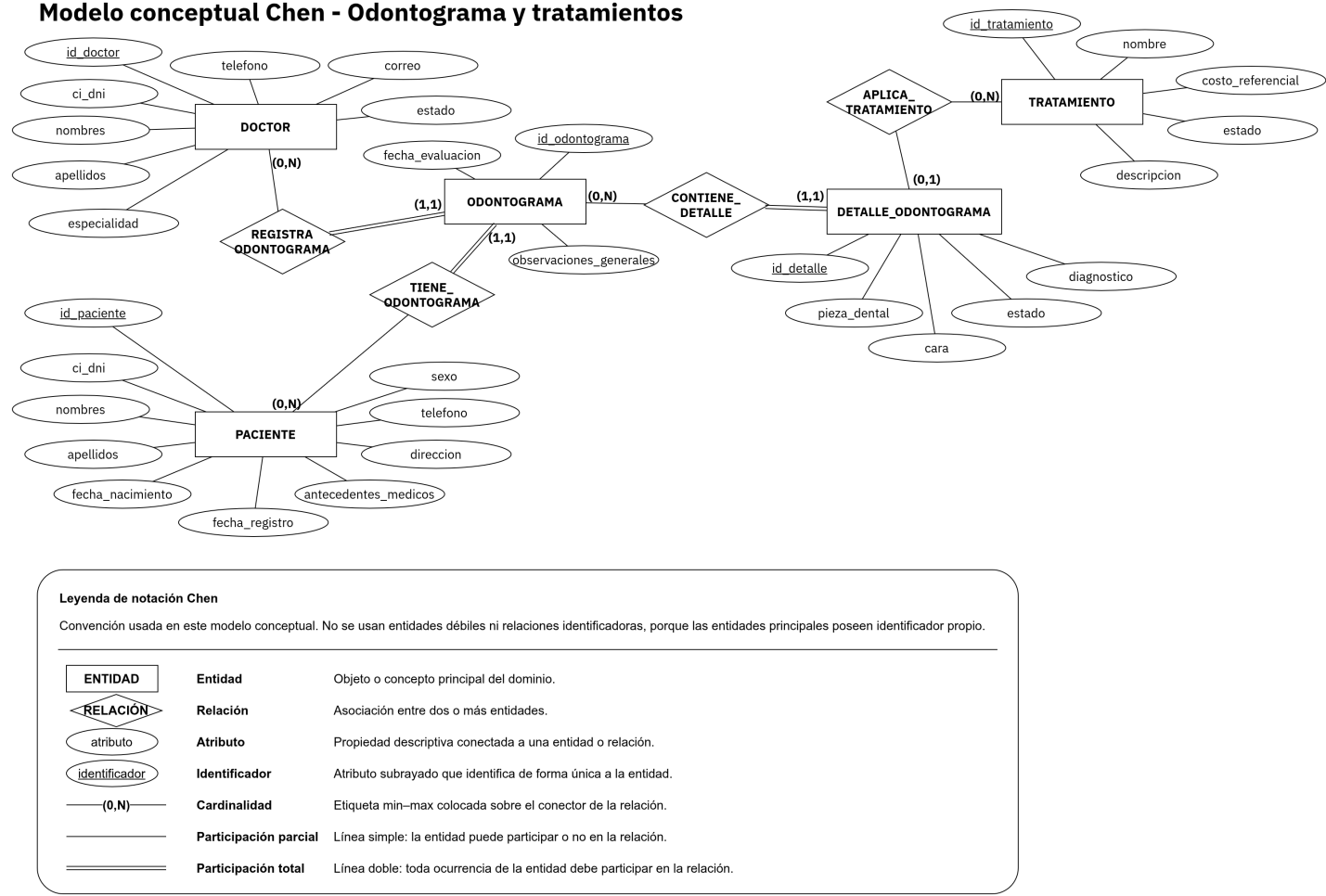


Figura 3

Modelo conceptual Chen: odontograma y tratamientos.

Modelo conceptual Chen - Finanzas y pagos

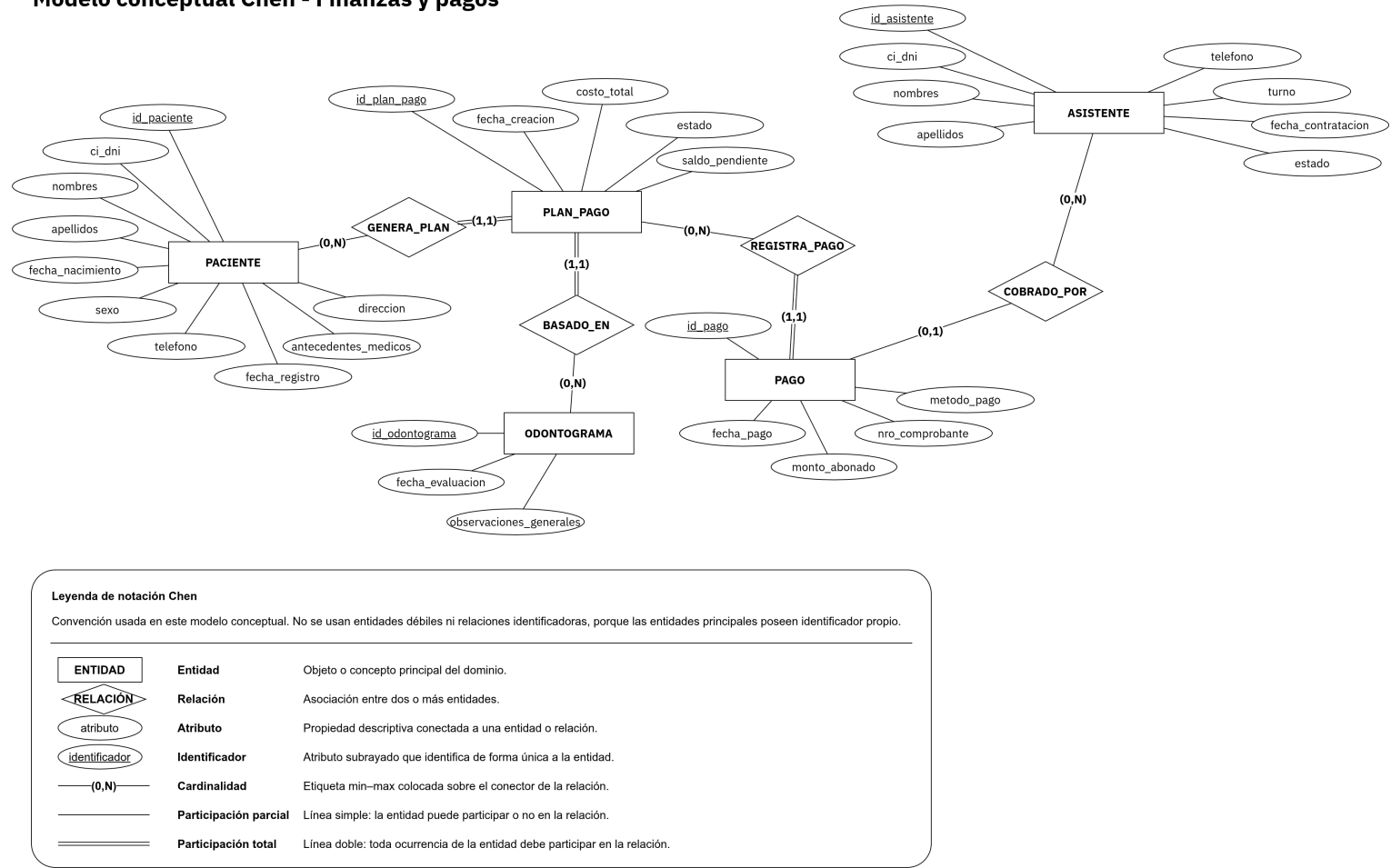


Figura 4

Modelo conceptual Chen: finanzas y pagos.

Modelo conceptual Chen - Compras e inventario

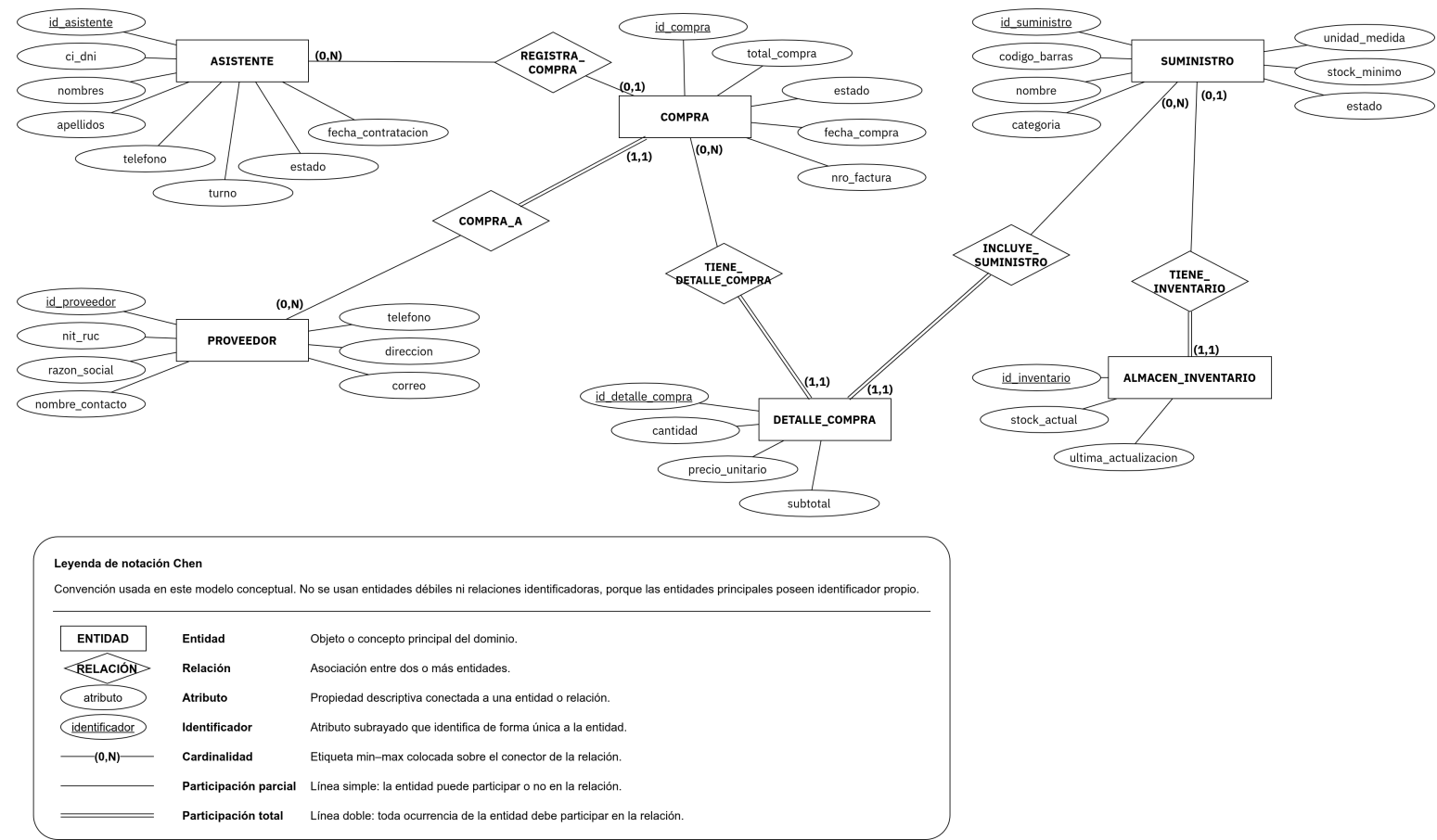


Figura 5

Modelo conceptual Chen: compras e inventario.

Transformación al modelo relacional

La transformación del modelo conceptual al modelo relacional siguió reglas estándar de diseño de bases de datos. Cada entidad fuerte se transformó en una tabla, los identificadores se transformaron en claves primarias y las relaciones uno a muchos se materializaron mediante claves foráneas en el lado dependiente.

Las relaciones opcionales se representaron mediante columnas que aceptan valores nulos, mientras que las relaciones obligatorias se implementaron con claves foráneas NOT NULL. Las reglas de dominio simples se implementaron mediante restricciones CHECK, y las reglas operacionales se implementaron mediante triggers y procedimientos almacenados en Oracle 19c.

El resultado de la transformación produjo quince tablas relacionales, organizadas en los bloques de registro, atención clínica, finanzas e inventario.

Modelo lógico Crow's Foot

El modelo lógico en Crow's Foot fue generado en ER/Studio 2003. En este modelo se visualizan las tablas, claves primarias, claves foráneas, relaciones, cardinalidades y obligatoriedad.

Para la importación se preparó un DDL simplificado compatible con Oracle 11g, debido a que dicha versión era la plataforma Oracle más reciente disponible en ER/Studio 2003. Este DDL no reemplaza al diseño físico Oracle 19c, sino que facilita la representación lógica de tablas y relaciones.

El modelo contiene las quince tablas del diseño relacional. Se observan las relaciones entre pacientes, citas, odontogramas, pagos, compras e inventario. Las claves foráneas importadas permiten visualizar la cardinalidad entre tablas padre e hija.

Diseño físico en Oracle 19c

La implementación física se realizó en Oracle Database 19c dentro del schema FARMACIAS_BD3. En Oracle, el schema está asociado al usuario propietario de los objetos, por lo que se creó un usuario específico para contener las tablas, vistas, triggers y procedimientos del proyecto.

Scripts implementados

Tabla 1

Scripts físicos Oracle 19c

Script	Propósito
00_admin_crear_schema.sql	Crea el usuario/schema y asigna permisos.
01_limpieza_esquema.sql	Limpia objetos previos para permitir reejecución.
02_tablas.sql	Crea tablas, claves primarias, claves foráneas y restricciones.
03_vistas.sql	Crea la vista de historial clínico.
04_triggers.sql	Crea triggers de inventario y pagos.
05_procedimientos.sql	Define el procedimiento almacenado formal.
05_procedimientos_dbeaver.sql	Variante compatible con DBeaver.
06_datos_prueba.sql	Carga datos de prueba.

Objetos programables

Se implementó la vista VW_HISTORIAL_CLINICO, el trigger

TRG_ACTUALIZAR_STOCK_COMPRA, el trigger TRG_AUDITAR_PLAN_PAGO y el

procedimiento almacenado SP_REGISTRAR_PAGO. Estos objetos permiten centralizar reglas de consulta, inventario y pagos directamente en el motor de base de datos.

Diccionario de datos

El diccionario de datos completo se encuentra en el archivo 08_diccionario_datos/01_diccionario_datos.md. Este diccionario documenta cada tabla, campo, tipo de dato, nulabilidad, clave y descripción funcional.

Para evidenciar que el diccionario corresponde al esquema real, se ejecutó una consulta técnica sobre el catálogo de Oracle mediante USER_TAB_COLUMNS, USER_CONSTRAINTS y USER_CONS_COLUMNS. Esta validación permitió contrastar columnas, tipos de datos, restricciones y relaciones de claves foráneas.

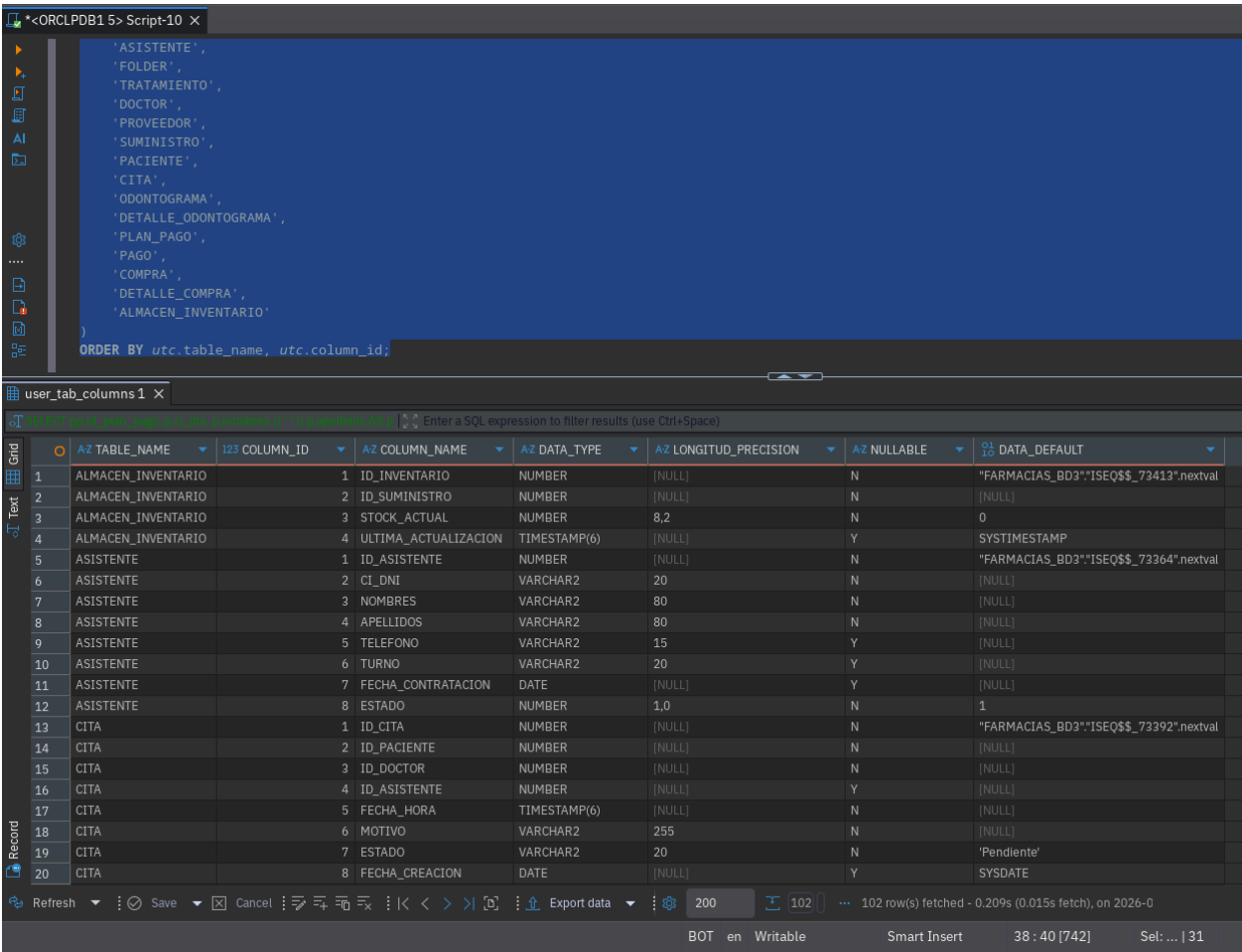


Figura 7
Extracción técnica de columnas desde Oracle.

SQL Developer Script window showing the query to extract foreign key relationships:

```
-- 4. Relaciones de claves foráneas
SELECT
  child.table_name AS tabla_hija,
  child_cols.column_name AS columna_hija,
  child.constraint_name AS fk_name,
  parent.table_name AS tabla_padre,
  parent_cols.column_name AS columna_padre,
  child.delete_rule
FROM user_constraints child
JOIN user_cons_columns child_cols
  ON child.constraint_name = child_cols.constraint_name
JOIN user_constraints parent
  ON child.r_constraint_name = parent.constraint_name
JOIN user_cons_columns parent_cols
  ON parent.constraint_name = parent_cols.constraint_name
  AND child_cols.position = parent_cols.position
WHERE child.constraint_type = 'R'
ORDER BY child.table_name, child.constraint_name, child_cols.position;
```

Results window showing the extracted relationships:

Child	AZ TABLA_HIJA	AZ COLUMNA_HIJA	AZ FK_NAME	AZ TABLA_PADRE	AZ COLUMNA_PADRE	AZ DELETE_RULE
1	ALMACEN_INVENTARIO	ID_SUMINISTRO	FK_INVENTARIO_SUMINISTRO	SUMINISTRO	ID_SUMINISTRO	NO ACTION
2	CITA	ID_ASISTENTE	FK_CITA_ASISTENTE	ASISTENTE	ID_ASISTENTE	NO ACTION
3	CITA	ID_DOCTOR	FK_CITA_DOCTOR	DOCTOR	ID_DOCTOR	NO ACTION
4	CITA	ID_PACIENTE	FK_CITA_PACIENTE	PACIENTE	ID_PACIENTE	NO ACTION
5	COMPRA	ID_ASISTENTE	FK_COMPRA_ASISTENTE	ASISTENTE	ID_ASISTENTE	NO ACTION
6	COMPRA	ID_PROVEEDOR	FK_COMPRA_PROVEEDOR	PROVEEDOR	ID_PROVEEDOR	NO ACTION
7	DETALLE_COMPRA	ID_COMPRA	FK_DETCOMPRA_COMPRA	COMPRA	ID_COMPRA	CASCADE
8	DETALLE_COMPRA	ID_SUMINISTRO	FK_DETCOMPRA_SUMINISTRO	SUMINISTRO	ID_SUMINISTRO	NO ACTION
9	DETALLE_ODONTOGRAMA	ID_ODONTOGRAMA	FK_DET_ODONTOGRAMA	ODONTOGRAMA	ID_ODONTOGRAMA	CASCADE
10	DETALLE_ODONTOGRAMA	ID_TRATAMIENTO	FK_DET_TRATAMIENTO	TRATAMIENTO	ID_TRATAMIENTO	NO ACTION
11	ODONTOGRAMA	ID_DOCTOR	FK_ODONTO_DOCTOR	DOCTOR	ID_DOCTOR	NO ACTION
12	ODONTOGRAMA	ID_PACIENTE	FK_ODONTO_PACIENTE	PACIENTE	ID_PACIENTE	NO ACTION
13	PACIENTE	ID_FOLDER	FK_PACIENTE_FOLDER	FOLDER	ID_FOLDER	SET NULL
14	PAGO	ID_ASISTENTE	FK_PAGO_ASISTENTE	ASISTENTE	ID_ASISTENTE	NO ACTION
15	PAGO	ID_PLAN_PAGO	FK_PAGO_PLAN	PLAN_PAGO	ID_PLAN_PAGO	NO ACTION
16	PLAN_PAGO	ID_ODONTOGRAMA	FK_PLAN_ODONTO	ODONTOGRAMA	ID_ODONTOGRAMA	NO ACTION
17	PLAN_PAGO	ID_PACIENTE	FK_PLAN_PACIENTE	PACIENTE	ID_PACIENTE	NO ACTION

Figura 8

Extracción técnica de relaciones desde Oracle.

Validaciones

Las validaciones se ejecutaron sobre Oracle Database 19c mediante DBeaver.

Primero se validó la creación del schema, luego la creación de tablas, restricciones, vista, triggers y procedimiento. Posteriormente, se cargaron datos de prueba y se verificó el comportamiento del inventario y del registro de pagos.

The screenshot shows the Oracle SQL Developer interface. The top window, titled '*<ORCLPDB1 5> Script-10 X', contains the following SQL query:

```
SELECT table_name
FROM user_tables
ORDER BY table_name;
```

The bottom window, titled 'user_tables 1 X', displays the results of the query in a grid view. The grid has a header row 'AZ TABLE_NAME' and 15 data rows. The data represents the tables in the schema FARMACIAS_BD3.

	AZ TABLE_NAME
1	ALMACEN_INVENTARIO
2	ASISTENTE
3	CITA
4	COMPRA
5	DETALLE_COMPRA
6	DETALLE_ODONTOGRAMA
7	DOCTOR
8	FOLDER
9	ODONTOGRAMA
10	PACIENTE
11	PAGO
12	PLAN_PAGO
13	PROVEEDOR
14	SUMINISTRO
15	TRATAMIENTO

The bottom status bar shows 'Refresh', 'Save', 'Cancel', 'Export data', and a page size of 200.

Figura 9

Tablas creadas en el schema FARMACIAS_BD3.

The screenshot displays the Oracle SQL Developer interface. The top pane shows a SQL script named 'Script-10' with the following query:

```
SELECT object_name, object_type, status
FROM user_objects
WHERE object_type IN ('TRIGGER', 'PROCEDURE', 'VIEW')
ORDER BY object_type, object_name;
```

The bottom pane shows the results of the query in a table grid. The table has three columns: 'AZ OBJECT_NAME', 'AZ OBJECT_TYPE', and 'AZ STATUS'. The results are as follows:

	AZ OBJECT_NAME	AZ OBJECT_TYPE	AZ STATUS
1	TRG_ACTUALIZAR_STOCK_COMPRA	TRIGGER	VALID
2	TRG_AUDITAR_PLAN_PAGO	TRIGGER	VALID
3	VW_HISTORIAL_CLINICO	VIEW	VALID

The interface also includes a sidebar on the left with icons for various database operations and a bottom toolbar with buttons for Refresh, Save, Cancel, and Export data.

Figura 10

Vista y triggers validados en Oracle.

The screenshot displays the Oracle SQL Developer environment. The top pane, titled '*<ORCLPDB1 5> Script-10 X', contains the following SQL query:

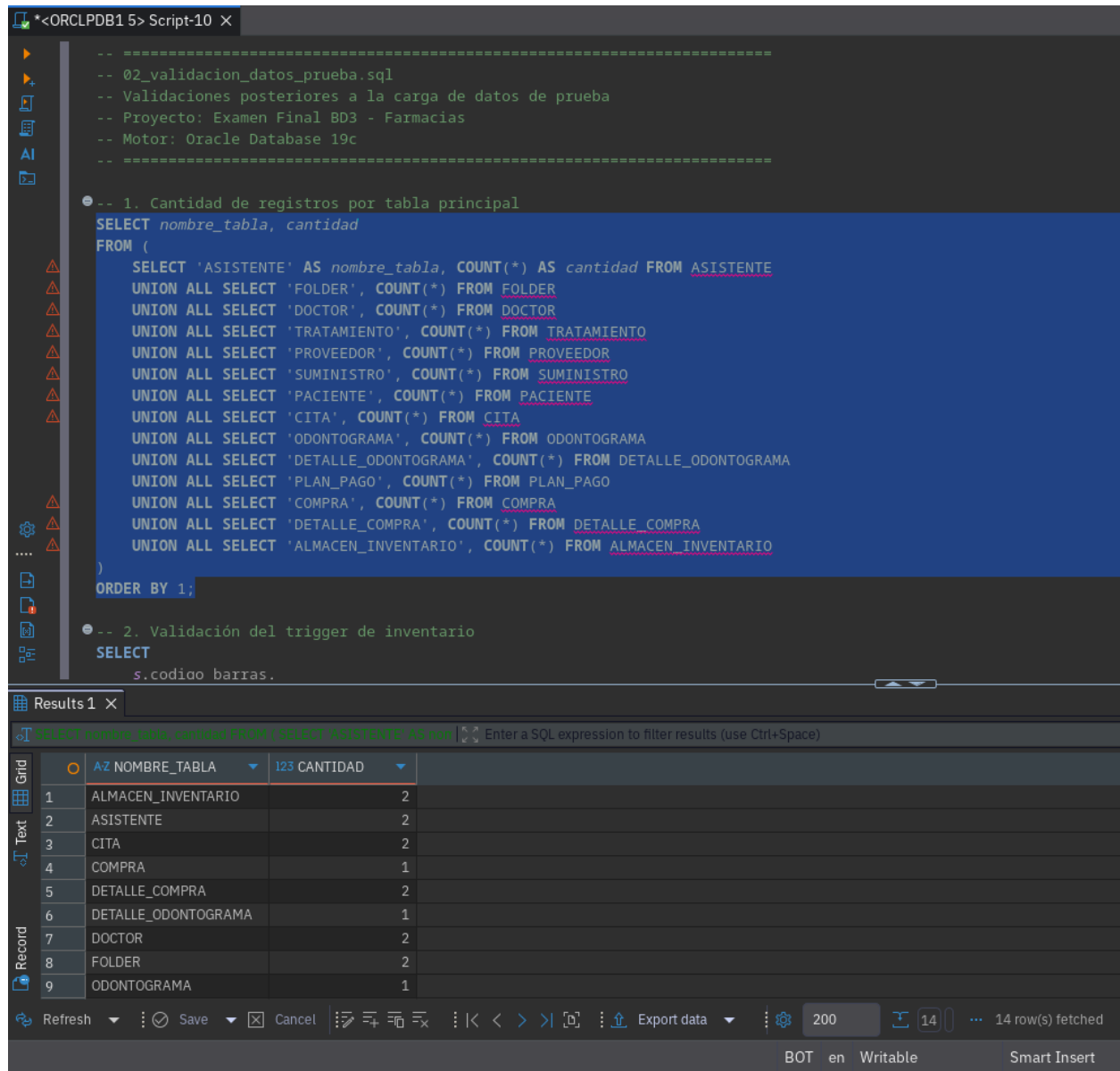
```
SELECT object_name, object_type, status
FROM user_objects
WHERE object_name = 'SP_REGISTRAR_PAGO';
```

The bottom pane, titled 'user_objects 1 X', shows the results of the query in a grid view. The grid has three columns: 'AZ OBJECT_NAME', 'AZ OBJECT_TYPE', and 'AZ STATUS'. The first row contains the data: 'SP_REGISTRAR_PAGO', 'PROCEDURE', and 'VALID'. The grid view includes a toolbar with options like 'Refresh', 'Save', 'Cancel', and 'Export data'. The bottom right corner shows a page number '200' and a record count '1'.

	AZ OBJECT_NAME	AZ OBJECT_TYPE	AZ STATUS
1	SP_REGISTRAR_PAGO	PROCEDURE	VALID

Figura 11

Procedimiento almacenado validado en Oracle.



The screenshot displays the Oracle SQL Developer environment. The top pane shows a script named 'Script-10' with the following content:

```
-- *****
-- 02_validacion_datos_prueba.sql
-- Validaciones posteriores a la carga de datos de prueba
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- *****

-- 1. Cantidad de registros por tabla principal
SELECT nombre_tabla, cantidad
FROM (
  SELECT 'ASISTENTE' AS nombre_tabla, COUNT(*) AS cantidad FROM ASISTENTE
  UNION ALL SELECT 'FOLDER', COUNT(*) FROM FOLDER
  UNION ALL SELECT 'DOCTOR', COUNT(*) FROM DOCTOR
  UNION ALL SELECT 'TRATAMIENTO', COUNT(*) FROM TRATAMIENTO
  UNION ALL SELECT 'PROVEEDOR', COUNT(*) FROM PROVEEDOR
  UNION ALL SELECT 'SUMINISTRO', COUNT(*) FROM SUMINISTRO
  UNION ALL SELECT 'PACIENTE', COUNT(*) FROM PACIENTE
  UNION ALL SELECT 'CITA', COUNT(*) FROM CITA
  UNION ALL SELECT 'ODONTOGRAMA', COUNT(*) FROM ODONTOGRAMA
  UNION ALL SELECT 'DETALLE_ODONTOGRAMA', COUNT(*) FROM DETALLE_ODONTOGRAMA
  UNION ALL SELECT 'PLAN_PAGO', COUNT(*) FROM PLAN_PAGO
  UNION ALL SELECT 'COMPRA', COUNT(*) FROM COMPRA
  UNION ALL SELECT 'DETALLE_COMPRA', COUNT(*) FROM DETALLE_COMPRA
  UNION ALL SELECT 'ALMACEN_INVENTARIO', COUNT(*) FROM ALMACEN_INVENTARIO
)
ORDER BY 1;

-- 2. Validación del trigger de inventario
SELECT
  s.codigo barras.
```

The bottom pane shows the 'Results 1' window with a grid view of the query results. The grid has two columns: 'NOMBRE_TABLA' and 'CANTIDAD'. The results are as follows:

	NOMBRE_TABLA	CANTIDAD
1	ALMACEN_INVENTARIO	2
2	ASISTENTE	2
3	CITA	2
4	COMPRA	1
5	DETALLE_COMPRA	2
6	DETALLE_ODONTOGRAMA	1
7	DOCTOR	2
8	FOLDER	2
9	ODONTOGRAMA	1

The bottom status bar indicates '14 row(s) fetched' and 'BOT en Writable Smart Insert'.

Figura 12

Validación de datos de prueba cargados.

The screenshot displays the Oracle SQL Developer environment. The top pane shows a script editor with the following SQL code:

```

p_metodo_pago => 'Efectivo',
p_comprobante => 'REC-001'
);
END;

-- Pagos registrados
SELECT
  pg.id_pago,
  pg.id_plan_pago,
  pg.id_asistente,
  pg.monto_abonado,
  pg.metodo_pago,
  pg.nro_comprobante,
  pg.fecha_pago
FROM PAGO pg
ORDER BY pg.id_pago;

-- Saldo después del pago
SELECT
  pp.id_plan_pago,
  p.ci_dni,
  p.nombres || ' ' || p.apellidos AS paciente,
  pp.costo_total,
  pp.saldo_pendiente,
  pp.estado
FROM PLAN_PAGO pp
JOIN PACIENTE p
  ON pp.id_paciente = p.id_paciente
WHERE p.ci_dni = 'P001';

```

The bottom pane shows the query result grid for the second query. The grid has the following columns: ID_PLAN_PAGO, CI_DNI, PACIENTE, COSTO_TOTAL, SALDO_PENDIENTE, and ESTADO. The first row of data is highlighted.

ID_PLAN_PAGO	CI_DNI	PACIENTE	COSTO_TOTAL	SALDO_PENDIENTE	ESTADO
1	P001	Daniel Choque Lima	180	100	Vigente

The status bar at the bottom indicates "1 row(s) fetched - 0.002s, o" and "BOT en Writable Smart Insert".

Figura 13

Prueba funcional del procedimiento de pago.

Comparación MySQL/Laravel vs Oracle 19c

La tesis base documenta un sistema web desarrollado con Laravel, MySQL y arquitectura MVC. En dicho enfoque, la base de datos funciona como repositorio persistente para una aplicación web. En la transformación realizada en este proyecto, Oracle 19c cumple un rol más estructural, porque concentra

restricciones, reglas de negocio y objetos programables dentro del motor de base de datos.

Tabla 2

Comparación entre el enfoque MySQL/Laravel y Oracle 19c

Criterio	MySQL/Laravel	Oracle 19c
Enfoque	Sistema web completo.	Base de datos documentada y validada.
Arquitectura	MVC con Laravel.	Modelo relacional con SQL y PL/SQL.
Identificadores	Enteros autoincrementales o equivalentes.	NUMBERGENERATEDALWAYSASIDENTITY.
Texto	VARCHAR.	VARCHAR2.
Fechas	DATE, DATETIME y TIMESTAMP.	DATE, TIMESTAMP, SYSDATE y SYSTIMESTAMP.
Montos	DECIMAL.	NUMBER(10,2).
Reglas de negocio	Principalmente desde aplicación y base de datos.	Restricciones, triggers y procedimientos en el motor.
Inventario	Lógica funcional del sistema.	Trigger de actualización automática de stock.
Pagos	Módulo de pagos.	Procedimiento almacenado y control de saldo.
Evidencia	Interfaces y funcionamiento del sistema.	Scripts, validaciones y catálogo Oracle.

La transformación no consiste únicamente en cambiar tipos de datos. También formaliza reglas de integridad, separa responsabilidades y permite validar el diseño desde el propio motor Oracle.

Fuentes del trabajo

En este informe, el término fuentes se refiere a los archivos, diagramas, scripts y evidencias generados durante el desarrollo del proyecto. Estos artefactos se encuentran organizados dentro del repositorio del examen final.

Tabla 3

Fuentes y artefactos del trabajo

Categoría	Archivo o evidencia
Requisitos	00_requisitos/01_alcance_y_supuestos.md
Modelo Chen editable	01_conceptual_chen/modelo_conceptual_chen_bd3.drawio
Modelo Chen exportado	01_conceptual_chen/modelo_chen_*.png
Transformación lógica	02_transformacion_logica/01_transformacion_modelo_relacional.md
Modelo Crow's Foot	03_crows_foot/modelo_logico_crows_foot_bd3.DM1
Modelo Crow's Foot exportado	03_crows_foot/modelo_logico_crows_foot_bd3.jpg
DDL y PL/SQL Oracle	05_oracle_ddl/*.sql
Evidencias	06_evidencias/*.png
Diccionario de datos	08_diccionario_datos/01_diccionario_datos.md
Comparación MySQL/Oracle	09_comparacion_mysql_oracle/*.md
Validaciones	10_validaciones/*.sql

Conclusiones

Se diseñó e implementó una base de datos relacional para procesos odontológicos, cubriendo registro de pacientes, citas, odontogramas, tratamientos, pagos, compras e inventario.

El modelo conceptual Chen permitió representar el dominio sin depender de un motor físico. La transformación relacional permitió derivar tablas, claves y relaciones. El modelo lógico Crow's Foot facilitó la visualización de la estructura relacional, mientras que Oracle Database 19c permitió implementar el diseño físico con restricciones, vista, triggers y procedimiento almacenado.

Las validaciones realizadas demostraron que los objetos fueron creados correctamente y que las reglas implementadas funcionan. En particular, se verificó la actualización de inventario al registrar compras y la actualización de saldo al registrar pagos mediante el procedimiento SP_REGISTRAR_PAGO.

Finalmente, la comparación con el enfoque MySQL/Laravel descrito en la tesis base permitió evidenciar que Oracle 19c ofrece una implementación más centrada en reglas del motor, integridad relacional explícita y validación técnica mediante el catálogo de base de datos.

Apéndice A

Anexo: scripts SQL

Creación de schema/usuario

DDL físico Oracle 19c

```
-- =====
-- 02_tablas.sql
-- Creación de tablas, claves primarias, claves foráneas y restricciones
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- =====

CREATE TABLE ASISTENTE (
    id_asistente NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ci_dni VARCHAR2(20) UNIQUE NOT NULL,
    nombres VARCHAR2(80) NOT NULL,
    apellidos VARCHAR2(80) NOT NULL,
    telefono VARCHAR2(15),
    turno VARCHAR2(20),
    fecha_contratacion DATE,
    estado NUMBER(1) DEFAULT 1 NOT NULL
);

CREATE TABLE FOLDER (
    id_folder NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo_archivo VARCHAR2(20) UNIQUE NOT NULL,
    estante VARCHAR2(20),
    seccion VARCHAR2(20),
    observaciones VARCHAR2(255)
);

CREATE TABLE TRATAMIENTO (
```

```
id_tratamiento NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
nombre VARCHAR2(100) NOT NULL,  
descripcion VARCHAR2(255),  
costo_referencial NUMBER(10, 2) NOT NULL,  
estado NUMBER(1) DEFAULT 1 NOT NULL  
);
```

```
CREATE TABLE DOCTOR (  
id_doctor NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
ci_dni VARCHAR2(20) UNIQUE NOT NULL,  
nombres VARCHAR2(80) NOT NULL,  
apellidos VARCHAR2(80) NOT NULL,  
especialidad VARCHAR2(100),  
telefono VARCHAR2(15),  
correo VARCHAR2(100) UNIQUE,  
estado NUMBER(1) DEFAULT 1 NOT NULL  
);
```

```
CREATE TABLE PROVEEDOR (  
id_proveedor NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
nit_ruc VARCHAR2(20) UNIQUE NOT NULL,  
razon_social VARCHAR2(100) NOT NULL,  
nombre_contacto VARCHAR2(80),  
telefono VARCHAR2(15),  
direccion VARCHAR2(255),  
correo VARCHAR2(100)  
);
```

```
CREATE TABLE SUMINISTRO (  
id_suministro NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
codigo_barras VARCHAR2(50) UNIQUE,  
nombre VARCHAR2(100) NOT NULL,  
categoria VARCHAR2(50),
```



```
    unidad_medida VARCHAR2(20),
    stock_minimo NUMBER(5) DEFAULT 5 NOT NULL,
    estado NUMBER(1) DEFAULT 1 NOT NULL
);

CREATE TABLE PACIENTE (
    id_paciente NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_folder NUMBER,
    ci_dni VARCHAR2(20) UNIQUE NOT NULL,
    nombres VARCHAR2(80) NOT NULL,
    apellidos VARCHAR2(80) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    sexo CHAR(1),
    telefono VARCHAR2(15),
    direccion VARCHAR2(255),
    antecedentes_medicos VARCHAR2(500),
    fecha_registro DATE DEFAULT SYSDATE,
    CONSTRAINT fk_paciente_folder FOREIGN KEY (id_folder)
        REFERENCES FOLDER(id_folder) ON DELETE SET NULL
);

CREATE TABLE CITA (
    id_cita NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_paciente NUMBER NOT NULL,
    id_doctor NUMBER NOT NULL,
    id_asistente NUMBER,
    fecha_hora TIMESTAMP NOT NULL,
    motivo VARCHAR2(255) NOT NULL,
    estado VARCHAR2(20) DEFAULT 'Pendiente' NOT NULL,
    fecha_creacion DATE DEFAULT SYSDATE,
    CONSTRAINT fk_cita_paciente FOREIGN KEY (id_paciente)
        REFERENCES PACIENTE(id_paciente),
    CONSTRAINT fk_cita_doctor FOREIGN KEY (id_doctor)
```

```
REFERENCES DOCTOR(id_doctor),
CONSTRAINT fk_cita_asistente FOREIGN KEY (id_asistente)
REFERENCES ASISTENTE(id_asistente),
CONSTRAINT chk_estado_cita
CHECK (estado IN ('Pendiente', 'Confirmada', 'Atendida', 'Cancelada',
'Reprogramada'))
);

CREATE TABLE ODONTOGRAMA (
    id_odontograma NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_paciente NUMBER NOT NULL,
    id_doctor NUMBER NOT NULL,
    fecha_evaluacion DATE DEFAULT SYSDATE NOT NULL,
    observaciones_generales VARCHAR2(500),
CONSTRAINT fk_odonto_paciente FOREIGN KEY (id_paciente)
    REFERENCES PACIENTE(id_paciente),
CONSTRAINT fk_odonto_doctor FOREIGN KEY (id_doctor)
    REFERENCES DOCTOR(id_doctor)
);

CREATE TABLE DETALLE_ODONTOGRAMA (
    id_detalle NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_odontograma NUMBER NOT NULL,
    id_tratamiento NUMBER,
    pieza_dental VARCHAR2(10) NOT NULL,
    cara VARCHAR2(20) NOT NULL,
    diagnostico VARCHAR2(100) NOT NULL,
    estado VARCHAR2(20) DEFAULT 'Por tratar',
CONSTRAINT fk_det_odontograma FOREIGN KEY (id_odontograma)
    REFERENCES ODONTOGRAMA(id_odontograma) ON DELETE CASCADE,
CONSTRAINT fk_det_tratamiento FOREIGN KEY (id_tratamiento)
    REFERENCES TRATAMIENTO(id_tratamiento)
);
```

```
CREATE TABLE PLAN_PAGO (  
    id_plan_pago NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_paciente NUMBER NOT NULL,  
    id_odontograma NUMBER NOT NULL,  
    fecha_creacion DATE DEFAULT SYSDATE,  
    costo_total NUMBER(10, 2) NOT NULL,  
    saldo_pendiente NUMBER(10, 2) NOT NULL,  
    estado VARCHAR2(20) DEFAULT 'Vigente',  
    CONSTRAINT fk_plan_paciente FOREIGN KEY (id_paciente)  
        REFERENCES PACIENTE(id_paciente),  
    CONSTRAINT fk_plan_odonto FOREIGN KEY (id_odontograma)  
        REFERENCES ODONTOGRAMA(id_odontograma)  
);
```

```
CREATE TABLE PAGO (  
    id_pago NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_plan_pago NUMBER NOT NULL,  
    id_asistente NUMBER,  
    fecha_pago TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,  
    monto_abonado NUMBER(10, 2) NOT NULL,  
    metodo_pago VARCHAR2(30) NOT NULL,  
    nro_comprobante VARCHAR2(50),  
    CONSTRAINT fk_pago_plan FOREIGN KEY (id_plan_pago)  
        REFERENCES PLAN_PAGO(id_plan_pago),  
    CONSTRAINT fk_pago_asistente FOREIGN KEY (id_asistente)  
        REFERENCES ASISTENTE(id_asistente),  
    CONSTRAINT chk_monto_pago CHECK (monto_abonado > 0)  
);
```

```
CREATE TABLE COMPRA (  
    id_compra NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_proveedor NUMBER NOT NULL,
```

```
id_asistente NUMBER,
fecha_compra DATE NOT NULL,
nro_factura VARCHAR2(50),
total_compra NUMBER(10, 2) NOT NULL,
estado VARCHAR2(20) DEFAULT 'Completada',
CONSTRAINT fk_compra_proveedor FOREIGN KEY (id_proveedor)
    REFERENCES PROVEEDOR(id_proveedor),
CONSTRAINT fk_compra_asistente FOREIGN KEY (id_asistente)
    REFERENCES ASISTENTE(id_asistente)
);

CREATE TABLE DETALLE_COMPRA (
    id_detalle_compra NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_compra NUMBER NOT NULL,
    id_suministro NUMBER NOT NULL,
    cantidad NUMBER(8,2) NOT NULL,
    precio_unitario NUMBER(10, 2) NOT NULL,
    subtotal NUMBER(10, 2) NOT NULL,
    CONSTRAINT fk_detcompra_compra FOREIGN KEY (id_compra)
        REFERENCES COMPRA(id_compra) ON DELETE CASCADE,
    CONSTRAINT fk_detcompra_suministro FOREIGN KEY (id_suministro)
        REFERENCES SUMINISTRO(id_suministro)
);

CREATE TABLE ALMACEN_INVENTARIO (
    id_inventario NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_suministro NUMBER UNIQUE NOT NULL,
    stock_actual NUMBER(8,2) DEFAULT 0 NOT NULL,
    ultima_actualizacion TIMESTAMP DEFAULT SYSTIMESTAMP,
    CONSTRAINT fk_inventario_suministro FOREIGN KEY (id_suministro)
        REFERENCES SUMINISTRO(id_suministro)
);
```

Vista

```
-- =====
-- 03_vistas.sql
-- Vistas gerenciales y de consulta
-- =====

CREATE OR REPLACE VIEW VW_HISTORIAL_CLINICO AS
SELECT
    p.ci_dni AS dni_paciente,
    p.nombres || ' ' || p.apellidos AS nombre_paciente,
    d.nombres || ' ' || d.apellidos AS doctor_tratante,
    o.fecha_evaluacion,
    det.pieza_dental,
    det.diagnostico,
    t.nombre AS tratamiento_aplicado,
    det.estado AS estado_tratamiento
FROM PACIENTE p
JOIN ODONTOGRAMA o ON p.id_paciente = o.id_paciente
JOIN DOCTOR d ON o.id_doctor = d.id_doctor
JOIN DETALLE_ODONTOGRAMA det ON o.id_odontograma = det.id_odontograma
LEFT JOIN TRATAMIENTO t ON det.id_tratamiento = t.id_tratamiento;
```

Triggers

```
-- =====
-- 04_triggers.sql
-- Disparadores de automatización e integridad operacional
-- =====

CREATE OR REPLACE TRIGGER TRG_ACTUALIZAR_STOCK_COMPRA
AFTER INSERT ON DETALLE_COMPRA
FOR EACH ROW
BEGIN
```

```
UPDATE ALMACEN_INVENTARIO
SET stock_actual = stock_actual + :NEW.cantidad,
    ultima_actualizacion = SYSTIMESTAMP
WHERE id_suministro = :NEW.id_suministro;

IF SQL%ROWCOUNT = 0 THEN
    INSERT INTO ALMACEN_INVENTARIO (id_suministro, stock_actual)
    VALUES (:NEW.id_suministro, :NEW.cantidad);
END IF;
END;
/

CREATE OR REPLACE TRIGGER TRG_AUDITAR_PLAN_PAGO
BEFORE UPDATE OF saldo_pendiente ON PLAN_PAGO
FOR EACH ROW
BEGIN
    IF :NEW.saldo_pendiente < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: El saldo pendiente no puede
        ser menor a cero.');
```

```
    ELSIF :NEW.saldo_pendiente = 0 THEN
        :NEW.estado := 'Pagado';
    END IF;
END;
/
```

Procedimiento almacenado

```
-- =====
-- 05_procedimientos.sql
-- Procedimientos almacenados PL/SQL
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
--
```

```
-- Nota para DBeaver:
-- Seleccionar desde CREATE OR REPLACE PROCEDURE hasta END SP_REGISTRAR_PAGO;
-- y ejecutar con Ctrl + Enter. No seleccionar la barra final si DBeaver
-- presenta problemas con el delimitador de script.
-- =====

CREATE OR REPLACE PROCEDURE SP_REGISTRAR_PAGO (
    p_id_plan_pago IN NUMBER,
    p_id_asistente IN NUMBER,
    p_monto IN NUMBER,
    p_metodo_pago IN VARCHAR2,
    p_comprobante IN VARCHAR2
)
AS
    v_saldo_actual PLAN_PAGO.saldo_pendiente%TYPE;
BEGIN
    SELECT saldo_pendiente
    INTO v_saldo_actual
    FROM PLAN_PAGO
    WHERE id_plan_pago = p_id_plan_pago;

    IF p_monto <= 0 THEN
        RAISE_APPLICATION_ERROR(-20003, 'El monto del pago debe ser mayor a
cero.');
```

```
    END IF;

    IF p_monto > v_saldo_actual THEN
        RAISE_APPLICATION_ERROR(-20002, 'El monto a pagar excede el saldo
pendiente.');
```

```
    END IF;

    INSERT INTO PAGO (
        id_plan_pago,
```

```
        id_asistente,
        monto_abonado,
        metodo_pago,
        nro_comprobante
    )
    VALUES (
        p_id_plan_pago,
        p_id_asistente,
        p_monto,
        p_metodo_pago,
        p_comprobante
    );

    UPDATE PLAN_PAGO
    SET saldo_pendiente = saldo_pendiente - p_monto
    WHERE id_plan_pago = p_id_plan_pago;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004, 'El plan de pago especificado no
        existe.');
```

END SP_REGISTRAR_PAGO;

/

Datos de prueba

```
-- =====
-- 06_datos_prueba.sql
-- Carga de datos de prueba para validar el esquema Farmacias_BD3
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- =====
```



```
-- Limpieza lógica de datos, respetando dependencias
DELETE FROM PAGO;
DELETE FROM PLAN_PAGO;
DELETE FROM DETALLE_ODONTOGRAMA;
DELETE FROM ODONTOGRAMA;
DELETE FROM CITA;
DELETE FROM DETALLE_COMPRA;
DELETE FROM COMPRA;
DELETE FROM ALMACEN_INVENTARIO;
DELETE FROM PACIENTE;
DELETE FROM SUMINISTRO;
DELETE FROM PROVEEDOR;
DELETE FROM DOCTOR;
DELETE FROM TRATAMIENTO;
DELETE FROM FOLDER;
DELETE FROM ASISTENTE;

COMMIT;

-- Asistentes
INSERT INTO ASISTENTE (ci_dni, nombres, apellidos, telefono, turno,
    fecha_contratacion, estado)
VALUES ('A001', 'María', 'Quispe Flores', '70000001', 'Mañana', DATE '
    2026-01-10', 1);

INSERT INTO ASISTENTE (ci_dni, nombres, apellidos, telefono, turno,
    fecha_contratacion, estado)
VALUES ('A002', 'Luis', 'Mamani Rojas', '70000002', 'Tarde', DATE '2026-02-15'
    , 1);

-- Archivadores / folders
INSERT INTO FOLDER (codigo_archivo, estante, seccion, observaciones)
VALUES ('FOL-001', 'E1', 'A', 'Historia clínica activa');
```

```
INSERT INTO FOLDER (codigo_archivo, estante, seccion, observaciones)
VALUES ('FOL-002', 'E1', 'B', 'Historia clínica activa');

-- Doctores
INSERT INTO DOCTOR (ci_dni, nombres, apellidos, especialidad, telefono, correo
, estado)
VALUES ('D001', 'Carlos', 'Ramos Salas', 'Odontología general', '71000001', '
carlos.ramos@demo.test', 1);

INSERT INTO DOCTOR (ci_dni, nombres, apellidos, especialidad, telefono, correo
, estado)
VALUES ('D002', 'Andrea', 'Torres Vega', 'Ortodoncia', '71000002', 'andrea.
torres@demo.test', 1);

-- Tratamientos
INSERT INTO TRATAMIENTO (nombre, descripcion, costo_referencial, estado)
VALUES ('Limpieza dental', 'Profilaxis dental básica', 120.00, 1);

INSERT INTO TRATAMIENTO (nombre, descripcion, costo_referencial, estado)
VALUES ('Curación dental', 'Restauración con resina', 180.00, 1);

INSERT INTO TRATAMIENTO (nombre, descripcion, costo_referencial, estado)
VALUES ('Ortodoncia control', 'Control mensual de ortodoncia', 250.00, 1);

-- Proveedores
INSERT INTO PROVEEDOR (nit_ruc, razon_social, nombre_contacto, telefono,
direccion, correo)
VALUES ('PRV-001', 'Dental Supply SRL', 'Roxana Paredes', '72000001', 'Av.
Central 123', 'ventas@dentalsupply.test');

INSERT INTO PROVEEDOR (nit_ruc, razon_social, nombre_contacto, telefono,
direccion, correo)
```

```
VALUES ('PRV-002', 'Insumos Odonto Bolivia', 'Javier López', '72000002', '
    Calle Comercio 456', 'contacto@insumosodonto.test');

-- Suministros
INSERT INTO SUMINISTRO (codigo_barras, nombre, categoria, unidad_medida,
    stock_minimo, estado)
VALUES ('SUM-001', 'Guantes descartables', 'Bioseguridad', 'Caja', 5, 1);

INSERT INTO SUMINISTRO (codigo_barras, nombre, categoria, unidad_medida,
    stock_minimo, estado)
VALUES ('SUM-002', 'Anestesia dental', 'Medicamento', 'Unidad', 10, 1);

INSERT INTO SUMINISTRO (codigo_barras, nombre, categoria, unidad_medida,
    stock_minimo, estado)
VALUES ('SUM-003', 'Resina compuesta', 'Material odontológico', 'Unidad', 3,
    1);

-- Pacientes
INSERT INTO PACIENTE (
    id_folder, ci_dni, nombres, apellidos, fecha_nacimiento, sexo,
    telefono, direccion, antecedentes_medicos
)
SELECT
    f.id_folder, 'P001', 'Daniel', 'Choque Lima', DATE '1998-05-12', 'M',
    '73000001', 'Zona Norte 100', 'Sin antecedentes relevantes'
FROM FOLDER f
WHERE f.codigo_archivo = 'FOL-001';

INSERT INTO PACIENTE (
    id_folder, ci_dni, nombres, apellidos, fecha_nacimiento, sexo,
    telefono, direccion, antecedentes_medicos
)
SELECT
```

```
f.id_folder, 'P002', 'Lucía', 'Vargas Soto', DATE '2001-09-21', 'F',
'73000002', 'Zona Sur 200', 'Alergia declarada a penicilina'
FROM FOLDER f
WHERE f.codigo_archivo = 'FOL-002';

-- Citas
INSERT INTO CITA (id_paciente, id_doctor, id_asistente, fecha_hora, motivo,
estado)
SELECT p.id_paciente, d.id_doctor, a.id_asistente,
TIMESTAMP '2026-06-20 09:00:00',
'Evaluación inicial', 'Confirmada'
FROM PACIENTE p, DOCTOR d, ASISTENTE a
WHERE p.ci_dni = 'P001'
AND d.ci_dni = 'D001'
AND a.ci_dni = 'A001';

INSERT INTO CITA (id_paciente, id_doctor, id_asistente, fecha_hora, motivo,
estado)
SELECT p.id_paciente, d.id_doctor, a.id_asistente,
TIMESTAMP '2026-06-20 10:00:00',
'Control de ortodoncia', 'Pendiente'
FROM PACIENTE p, DOCTOR d, ASISTENTE a
WHERE p.ci_dni = 'P002'
AND d.ci_dni = 'D002'
AND a.ci_dni = 'A002';

-- Odontograma
INSERT INTO ODONTOGRAMA (id_paciente, id_doctor, fecha_evaluacion,
observaciones_generales)
SELECT p.id_paciente, d.id_doctor, DATE '2026-06-20', 'Evaluación inicial
registrada'
FROM PACIENTE p, DOCTOR d
WHERE p.ci_dni = 'P001'
```

```
    AND d.ci_dni = 'D001';

INSERT INTO DETALLE_ODONTOGRAMA (
    id_odontograma, id_tratamiento, pieza_dental, cara, diagnostico, estado
)
SELECT o.id_odontograma, t.id_tratamiento, '16', 'Oclusal', 'Caries inicial',
    'Por tratar'
FROM ODONTOGRAMA o
JOIN PACIENTE p ON o.id_paciente = p.id_paciente
JOIN TRATAMIENTO t ON t.nombre = 'Curación dental'
WHERE p.ci_dni = 'P001';

-- Plan de pago
INSERT INTO PLAN_PAGO (id_paciente, id_odontograma, fecha_creacion,
    costo_total, saldo_pendiente, estado)
SELECT p.id_paciente, o.id_odontograma, DATE '2026-06-20', 180.00, 180.00, '
    Vigente'
FROM PACIENTE p
JOIN ODONTOGRAMA o ON p.id_paciente = o.id_paciente
WHERE p.ci_dni = 'P001';

-- Compra y detalle de compra
INSERT INTO COMPRA (id_proveedor, id_asistente, fecha_compra, nro_factura,
    total_compra, estado)
SELECT pr.id_proveedor, a.id_asistente, DATE '2026-06-19', 'FAC-001', 450.00,
    'Completada'
FROM PROVEEDOR pr, ASISTENTE a
WHERE pr.nit_ruc = 'PRV-001'
    AND a.ci_dni = 'A001';

INSERT INTO DETALLE_COMPRA (id_compra, id_suministro, cantidad,
    precio_unitario, subtotal)
SELECT c.id_compra, s.id_suministro, 10, 25.00, 250.00
```

```
FROM COMPRA c, SUMINISTRO s
WHERE c.nro_factura = 'FAC-001'
      AND s.codigo_barras = 'SUM-001';

INSERT INTO DETALLE_COMPRA (id_compra, id_suministro, cantidad,
                             precio_unitario, subtotal)
SELECT c.id_compra, s.id_suministro, 5, 40.00, 200.00
FROM COMPRA c, SUMINISTRO s
WHERE c.nro_factura = 'FAC-001'
      AND s.codigo_barras = 'SUM-003';

COMMIT;
```

Apéndice B

*

Referencias

- Calvo Arteaga, C. A., & Cobos Vargas, I. J. (2025). Desarrollo de un sistema de información web para la administración de los procesos de registro, atención, inventario y finanzas del consultorio odontológico LALYSIDENT del distrito de Cusco [Tesis usada como referencia documental para el análisis del dominio y comparación MySQL/Laravel].
- Chen, P. P.-S. (1976). The Entity-Relationship Model: Toward a Unified View of Data.
- DBeaver. (2024). DBeaver Documentation [Documentación oficial de DBeaver para ejecución de scripts SQL]. <https://dbeaver.com/docs/dbeaver/>
- JGraph Ltd. (2024). diagrams.net Documentation [Documentación oficial de diagrams.net/draw.io]. <https://www.drawio.com/doc/>
- Oracle. (2024). Oracle Database 19c Documentation [Documentación oficial consultada para características generales de Oracle Database 19c]. <https://docs.oracle.com/en/database/oracle/oracle-database/19/>